πMath

Elaborato Esame di Stato 2020/2021

Rocca Patrick

I.T.T. Guglielmo Marconi
5A Informatica Big Data Analytics

Indice

1 Panoramica generale **2**
- 1.1 PiMath . . . 2
- 1.2 Possibili sviluppi futuri . . . 2

2 Architettura **4**
- 2.1 Architettura del progetto . . . 4
- 2.2 Architettura a microservizi . . . 5

3 Rete e protocolli **8**
- 3.1 Architettura di rete e DMZ . . . 8
- 3.2 HTTP . . . 9
- 3.3 HTTPS . . . 12

4 Microservizi e Server Spring Boot **13**
- 4.1 API per il calcolo matematico . . . 13
  - 4.1.1 Sympy . . . 14
  - 4.1.2 AWS Lambda . . . 15
  - 4.1.3 AWS Gateway . . . 16
- 4.2 API per l'interazione col database . . . 17
  - 4.2.1 Database . . . 18
  - 4.2.2 Server Spring Boot . . . 21

5 Client **28**
- 5.1 Angular . . . 28
  - 5.1.1 Component e Service . . . 28
  - 5.1.2 Directive . . . 30
  - 5.1.3 Data Binding . . . 31
- 5.2 Angular Material . . . 32
- 5.3 Login e Autenticazione . . . 33
- 5.4 Sezione di calcolo matematico . . . 35
  - 5.4.1 MathJax . . . 35
  - 5.4.2 Strumenti di calcolo . . . 36
- 5.5 Sezione di teoria . . . 37
  - 5.5.1 LaTeX.js . . . 37

6 Conclusioni e riflessioni **39**

7 Bibliografia **42**

1

Panoramica generale

1.1 PiMath

PiMath è un software per l'apprendimento della matematica, che mette a disposizione degli utenti strumenti di calcolo e capitoli di teoria sui più svariati argomenti. La sezione di calcolo comprende appunto vari strumenti dedicati alla soluzione di determinati quesiti matematici, dalla risoluzione di un'equazione a quella di una disequazione, dal calcolo di un limite a quello di una derivata, dalla soluzione di un integrale a quella di un'equazione differenziale. La sezione di teoria comprende invece una serie di macroargomenti del mondo della matematica, contenenti dei brevi capitoli di teoria o talvolta ulteriori sottoargomenti. In alcuni capitoli è inoltre possibile trovare dei link a delle esercitazioni python riguardanti l'argomento appositamente preparate su Colab.
PiMath prevede una serie di piani, con diverse fasce di prezzo, ognuno dei quali dà accesso a determinati strumenti matematici e a determinati macroargomenti di teoria.

1.2 Possibili sviluppi futuri

Un sicuro sviluppo futuro, che ho escluso dall'elaborato perchè lo ritengo al di fuori degli scopi dell'esame, è quello di progettare ed implementare un sistema di pagamento per i diversi piani, i quali sono al momento previsti per essere tutti disponibili gratuitamente.
Il software, come sarà illustrato più avanti, è progettato perchè sia semplice e veloce aggiungere nuovi servizi. Dunque un altro sicuro sviluppo futuro è quello che consiste

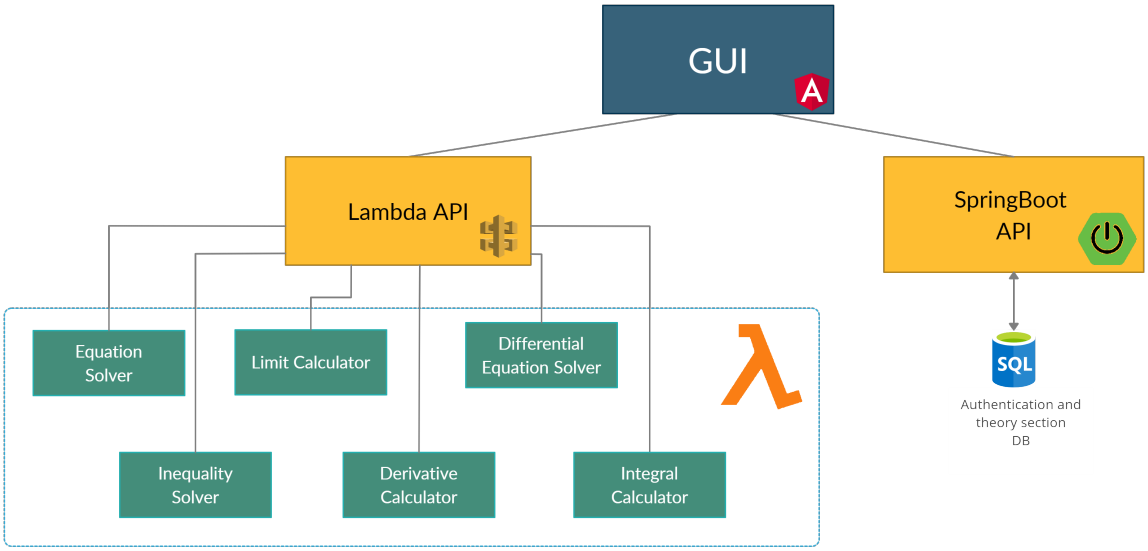
nell'aggiunta di nuovi strumenti di calcolo, ad esempio uno in grado di calcolare il valore a cui approccia una serie geometrica.
Un ultimo importante sviluppo futuro è la realizzazione di un software utile al gestore dell'applicazione, in questo caso quindi il sottoscritto, che permetta di inserire, modificare o eliminare piani o argomenti di teoria nel database con una semplice ed efficace interfaccia grafica (per la quale sono già predisposte delle API di inserimento/modifica/eliminazione), invece che andare a modificare direttamente la base di dati con delle query SQL come attualmente previsto. Questo software potrà inoltre includere uno strumento statistico, ancora utile al gestore dell'applicazione, che prevede l'analisi dei commenti di ogni capitolo (ogni utente può infatti inserire dei commenti ai vari capitoli di teoria) con un'intelligenza artificiale in grado di determinare sulla base di essi se il capitolo sia stato apprezzato o meno.

2

Architettura

2.1 Architettura del progetto

L'architettura del progetto è schematizzata nel seguente diagramma:



Essa si presenta come un misto tra una **SOA** (**Service Oriented Architecture**) e un'**architettura a microservizi**; è infatti composta da un'**interfaccia grafica**, realizzata in **Angular**, che fa parte contemporaneamente di due diverse architetture:

- L'architettura a microservizi, a sinistra, composta da tutta una serie di mi-

croservizi indipendenti realizzati in **Python** e caricati su **AWS Lambda** per la loro esecuzione. Essi, come si può dedurre dal nome dei singoli, implementano delle **funzioni per il calcolo matematico**, e sono esposti pubblicamente con delle **API** realizzate tramite **AWS Gateway**. Quest'ultime sono utilizzate dalla GUI per realizzare, per l'appunto, la parte di calcolo matematico.

- La SOA, a destra, composta da un **server Spring Boot** che interagisce direttamente con un **database relazionale** ed espone un'**API** utilizzabile dalla GUI. Grazie ad essa l'interfaccia grafica è in grado di interagire, indirettamente e secondo le regole dettate dal server Spring Boot, col database. Quest'API è utilizzata dalla GUI per l'**autenticazione degli utenti**, prevista infatti dal server in questione, e per l'esposizione al client **dei capitoli di teoria**, memorizzati assieme al resto dei **dati significativi dell'applicazione** all'interno del database.

2.2 Architettura a microservizi

Voglio soffermarmi in particolare sull'architettura a microservizi, piuttosto che andare nei dettagli della classica SOA, in quanto più moderna e sempre più utilizzata, e a mio avviso importante punto di forza del mio progetto.
L'architettura a microservizi, da me utilizzata come appena illustrato per la sezione di calcolo matematico, prevede una struttura formata da una serie di **unità indipendenti** che realizzano una determinata funzione, dette appunto **microservizi**, e un'**interfaccia grafica** che li utilizza per realizzare un'applicazione, con la quale comunicano attraverso delle **interfacce REST**. C'è inoltre la possibilità che un microservizio richieda delle funzioni già svolte da un altro microservizio per funzionare: in tal caso il servizio richiedente può comunicare con l'altro tramite l'interfaccia REST dell'ultimo ed utilizzare così le sue funzioni.
I vantaggi di quest'architettura sono molteplici, quelli più importanti sono:

- **Velocità e agilità di sviluppo**: la fase di sviluppo può essere suddivisa in diversi team che lavorano contemporaneamente ed indipendentemente ai diversi microservizi, velocizzando dunque in modo importante i tempi di costruzione del software e garantendo una notevole **semplicità nella gestione** delle varie unità, in quanto indipendenti tra di loro. Non è questo il mio caso in quanto ho lavorato al progetto da solo, ma se avessi avuto dei colleghi di lavoro ci saremmo potuti suddividere i vari microservizi per il calcolo matematico e svilupparli individualmente e indipendentemente, senza il rischio di incorrere poi problemi di compatibilità.

- **Scalabilità e flessibilità**: ogni microservizio può essere scalato o più in generale modificato senza influenzare gli altri, il che rende il processo notevolmente più rapido e semplice da gestire. Infatti se per esempio avessi da apportare una modifica al funzionamento interno dell'ipotetico microservizio per la soluzione di equazioni, potrei andare ad aggiornarlo senza dover minimamente modificare gli altri; e per di più non sarebbe nemmeno necessario aggiornare il client, a meno che non vengano apportate variazioni anche all'interfaccia del microservizio (modifica però che in genere non si attua).

- **Semplicità di distribuzione**: la pubblicazione di applicazioni e servizi è più rapida, in quanto per apportare modifiche a determinati aspetti è talvolta sufficiente andare a modificare il singolo microservizio, il cui deploy una volta modificato è notevolmente più semplice e rapido del deploy di un'intera applicazione. Se dopo la pubblicazione dell'applicazione volessi aggiungere un servizio di calcolo matematico, mi sarà sufficiente creare un altro microservizio dedicato, senza dover agire in nessuno modo sugli altri, per poi pubblicarlo ed eventualmente aggiungere al client un nuovo component Angular (andrò a spiegare i component più avanti) che lo utilizzi.

- **Libertà tecnologica**: per ogni microservizio possono essere scelti indipendentemente i **linguaggi** e le **tecnologie ottimali** per lo sviluppo, senza dover tener conto di quelli adottati dagli altri. Se il mio microservizio per la soluzione di equazioni fosse scritto in Python, non ci sarebbe nessun problema se per esempio quello per il calcolo delle derivate fosse scritto in JavaScript.

- **Riusabilità del codice**: Un singolo microservizio può essere utilizzato in **diverse applicazioni** che necessitano delle stesse funzioni, o da un altro microservizio stesso che per realizzare le sue funzionalità ha bisogno di determinate risorse già reperibili dal primo. Se eventualmente per un futuro servizio di calcolo serviranno delle funzioni già implementate da un altro, basterà richiamarlo con la sua interfaccia REST invece che scrivere nuovo codice.

- **Resilienza**: Errori o malfunzionamenti di determinati microservizi non compromettono il funzionamento degli altri, garantendo così la possibilità di lavorare a dei problemi di una determinata unità senza dover bloccare l'intera applicazione. Se ad esempio l'ipotetico servizio per il calcolo della derivate smettesse di funzionare, il client semplicemente sarà in grado di riconoscerlo e segnalare un errore su di esso, senza però influenzare il funzionamento degli altri.

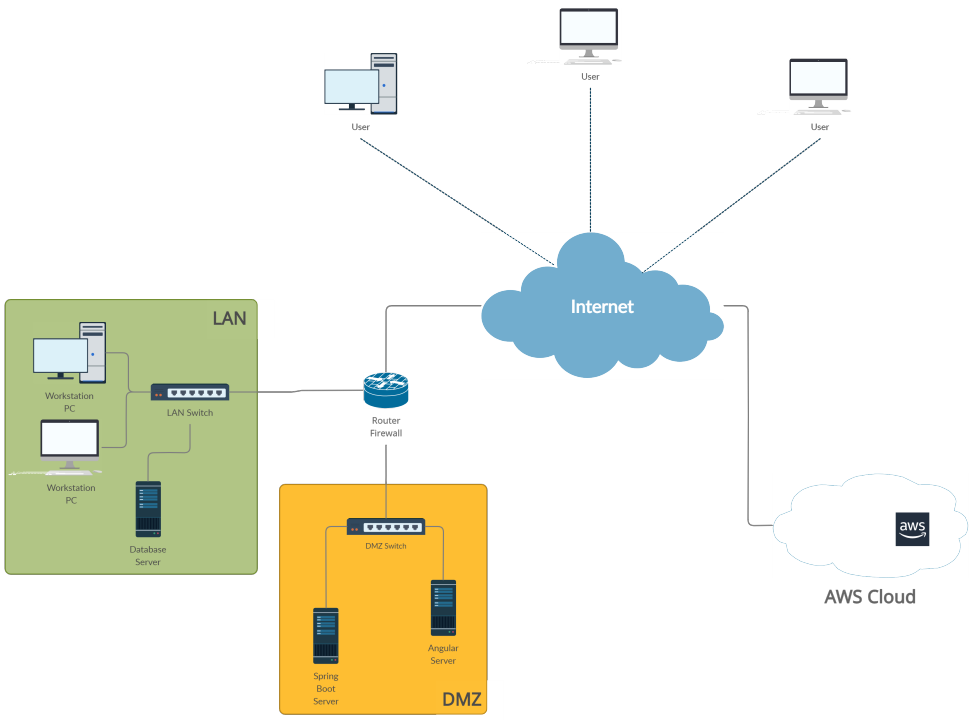
Ci sono ovviamente, come in ogni altra architettura, degli svantaggi. Uno dei più grandi è quello legato all'**efficienza**, non delle migliori in quanto microservizi e applicazioni devono continuamente scambiare messaggi in rete che possono rallentare il funzionamento del software. Inoltre si ha una certa **complessità di gestione delle comunicazioni** tra gli spesso numerosi componenti, e delle maggiori **difficoltà di test** per quanto riguarda la fase di integrazione dei microservizi.

3

Rete e protocolli

3.1 Architettura di rete e DMZ

La comunicazione tra i vari componenti del mio progetto funziona grazie alla sottostante architettura di rete, che permette grazie ai noti protocolli lo scambio di messaggi:



Questa rete può essere divisa in **3 principali componenti logiche**, interconnesse grazie ad Internet:

- Una **rete privata** dedicata ai server, composta da una **LAN** e una **DMZ**, all'interno della quale sono esposti i server.
- La **rete AWS** situata a Milano, contenente i **server AWS** sui quali verranno eseguiti i microservizi che andrò a creare per il calcolo matematico.
- L'**insieme degli utenti**, che possono raggiungere grazie ad Internet tutti i vari servizi cosicché l'applicazione possa funzionare.

DMZ

La prima delle sezioni indicate è, come detto, composta da una LAN e da una DMZ. Una DMZ, o **Demilitarized Zone**, è un'area in cui sia il traffico WAN sia quello LAN sono fortemente limitati e controllati da un **firewall**. Viene solitamente, e nel mio caso, utilizzata per permettere ai server posizionati al suo interno di fornire servizi all'esterno senza compromettere la **sicurezza della rete interna**, che nel mio caso deve tenere al sicuro il database. La DMZ all'interno della mia rete è realizzata sulla base del modello **Three Legged**, ovvero la rete è composta da un singolo firewall, incorporato nel router, avente tre porte: una connessa alla LAN, una alla WAN e una alla DMZ. Grazie a questa configurazione un utente esterno potrà raggiungere solo la DMZ, trovandosi in una sorta di vicolo cieco che gli impedirà di raggiungere altre aree della rete vicolo cieco.
Ha dunque senso che all'interno della LAN siano presenti il server nel quale risiede il database ed eventualmente qualche workstation utili per funzioni di debug dei server o della rete in generale, e che all'interno della DMZ siano presenti invece due macchine server, una dedicata all'esecuzione del server Spring Boot, e una dedicata all'esecuzione del server Angular, che esponga agli utenti il client, ovvero l'UI, dell'applicazione.

3.2 HTTP

Per la comunicazione in rete dei vari componenti è previsto l'utilizzo del protocollo **HTTPS**, alla cui base c'è il protocollo **HTTP**. HTTP, sigla di **HyperText Transfer Protocol**, è il protocollo di livello application più utilizzato per la trasmissione di informazioni sul web, basato sull'architettura client-server. Prevede infatti un server, che rimane in ascolto di eventuali messaggi da parte dei vari client: quando

una richiesta HTTP arriva al server, esso la processa per poi inviare un messaggio HTTP di risposta al client.
Un messaggio di richiesta è composto principalmente da 3 parti:

- La **riga di richiesta**, composta da **metodo**, **URI**, che specifica la destinazione, e **versione del protocollo**.
  Il metodo indica al server interrogato l'azione che il client si aspetta. Quelli da me utilizzati per la comunicazione tra il client Angular e i vari server sono il metodo **GET**, utilizzato principalmente quando si vogliono richiedere dei dati, e il metodo **POST**, utilizzato in primo luogo per l'invio di dati da salvare nel server. Ad esempio, per il recupero dei vari capitoli di teoria il client invierà al server Spring Boot dei messaggi HTTP con metodo GET, mentre per la registrazione verrà utilizzato il metodo POST, in quanto è previsto il salvataggio dei dati utente che si vogliono registrare sul server.

- Gli **header della richiesta**, ovvero dei dati utili principalmente al coordinamento tra client e server. Infatti grazie ad essi il client specifica tutta una serie di informazioni utili al server, come ad esempio il formato dei dati contenuti nella richiesta. Come spiegherò successivamente, per l'autenticazione della comunicazione tra il client Angular e il server Spring Boot verrà utilizzato il protocollo **HTTP Basic authentication** , che prevede l'utilizzo di un certo header per la trasmissione delle credenziali.

- Il **body** della richiesta, letteralmente il corpo del messaggio, che contiene i dati da trasmettere. Riprendendo l'esempio della registrazione, una richiesta di questo tipo conterrà all'interno del suo body tutti i dati dell'utente che si vogliono registrare.

Un messaggio di risposta è composto sostanzialmente dagli stessi campi di quello di richiesta, ad eccezione del primo. Ha dunque una **sezione di header**, un **body**, e, al posto della riga di richiesta, una **riga di stato**. La riga di stato contiene un codice a tre cifre che indica l'esito dell'elaborazione da parte del server. I più noti codici di risposta che ho previsto all'interno dei miei server sono il **200**, che indica l'esito positivo della richiesta, e il **400**, che indica un errore nella compilazione della richiesta. Un esempio di utilizzo del secondo si ha nella sezione di calcolo matematico: infatti se un utente inserisce in input un'espressione matematica contente degli errori sintattici il server risponderà con quel codice.

Un esempio di richiesta HTTP è quella per il recupero dei vari argomenti di teoria accessibili ad un certo utente:

```
GET /api/services/available HTTP/1.1

Host: localhost:8080
Connection: keep-alive
Access-Control-Allow-Origin: *
Accept: application/json, text/plain, */*
Authorization: Basic YWRtaW46Y205dmRBPT0=
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/91.0.4472.77 Safari/537.36 Edg/91.0.864.37
Origin: http://localhost:4200
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:4200/
Accept-Encoding: gzip, deflate, br
Accept-Language: it,it-IT;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6&%
```

In questa richiesta si può notare la riga di richiesta, all'inizio, e tutto l'insieme di header. Non c'è alcun body in quanto il messaggio rappresenta una richiesta di soli dati. Ad essa il server risponderà, in caso di esito positivo, col seguente messaggio:

```
HTTP/1.1 200

Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Access-Control-Allow-Origin: *
Set-Cookie: JSESSIONID=EACA22169CA17F8DF9E447C23D7E5FA3; Path=/; HttpOnly
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Fri, 28 May 2021 09:15:40 GMT
Keep-Alive: timeout=60
Connection: keep-alive
```

Qui si nota il codice di stato, affiancato alla versione HTTP e la lista di tutti gli header della risposta. Comprende inoltre un body, contenente i dati richiesti:

```
[
  {
    "id": 2,
    "name": "I numeri complessi",
    "description": "Il mondo dei numeri immaginari, la sua storia e tutte
       le sue conseguenze"
  },
  {
    "id": 3,
    "name": "Equazioni Differenziali",
    "description": "Le equazioni differenziali e la loro storia"
  }
]
```

3.3 HTTPS

HTTPS, siglia di **HTTP over Secure Socket Layer**, è un protocollo per la comunicazione sicura tramite Internet basato su HTTP. Consiste nell'utilizzo di HTTP all'interno di una connessione criptata tramite cittografia a chiave asimmetrica, e garantisce:

- L'**autenticazione** del server col quale si sta comunicando. I dati da esso ricevuti sono infatti firmati digitalmente, sulla base di un **certificato digitale** rilasciato da un ente certificatore riconosciuto. In questo modo al client è garantito che sia il server ad aver inviato i dati ricevuti e non un soggetto indesiderato che si è posto in mezzo alla comunicazione.
- La **protezione della privacy**. Infatti i messaggi trasmessi e ricevuti sono cifrati, e dunque non leggibili da un eventuale ascoltatore indesiderato.
- L'**integrità dei dati** scambiati. Eventuali tentativi di alterazione dei messaggi da parte di soggetti indesiderati saranno infatti riconosciuti.

Tutte le API che andrò a creare e il server Angular per la distribuzione del client dovranno comunicare in rete con HTTPS, così da avere una **sicurezza garantita** specialmente in fase di comunicazione con database contenente i dati privati dell'applicazione (come quelli sugli utenti).

4

Microservizi e Server Spring Boot

Andrò ora ad illustrare il **lato server** del progetto, suddiviso nella parte di **API per il calcolo matematico** e in quella di **server Spring Boot** che gestisce il database.

4.1 API per il calcolo matematico

Per questa sezione ho scelto di sviluppare dei **microservizi** esposti con delle **API RESTful** per il calcolo matematico, le quali saranno poi pubblicate cosicchè possano essere utilizzate da ogni programmatore che necessiti delle funzionalità disponibili.
I microservizi progettati, come si vede in figura nella sezione 2.1 sono 6:

- **Equation Solver**: risoluzione di una data equazione di qualsiasi tipo rispetto ad una data incognita.
- **Inequality Solver**: risoluzione di una data disequazione di qualiasi tipo rispetto ad una data incognita.
- **Limit Calculator**: calcolo del limite di una data funzione tendente ad un dato valore.
- **Derivative Calculator**: calcolo della derivata n-esima di una data funzione rispetto ad una data variabile.
- **Integral Calculator**: calcolo dell'integrale indefinito o, se un intervallo è specificato, definito, di una data funzione.
- **Differential Equation Solver**: risoluzione di una data equazione differenziale lineare di ordine non superiore al secondo.

Di queste API ho steso una documentazione (presente negli allegati), naturalmente in lingua inglese perchè possa arrivare al maggior numero di utenti possibili, che guidi il programmatore all'utilizzo dei servizi da me creati.
Quattro di questi sono già implementati (Equation Solver, Inequality Solver, Derivative Calculator, Differential Equation Solver) e pubblicati, quindi utilizzabili da chiunque.

Per sviluppare questa parte ho utilizzato diverse tecnologie, le cui 3 principali sono **SymPy**, **AWS Lambda** e **AWS Gateway**.

4.1.1 Sympy



Sympy è una **libreria open source** del linguaggio di programmazione **Python** per il calcolo simbolico. Essa espone dei metodi in grado di risolvere un grande varietà di quesiti matematici, inclusi tutti quelli prima elencati.
Ho quindi scelto il linguaggio di programmazione Python, versione 3.8, per la realizzazione dei vari microservizi, con l'utilizzo di questa libreria. Ad esempio per il servizio di calcolo della derivata di una funzione, che ho implementato, ho utilizzato il metodo `diff`:

```
derivative = function.diff((variable,order),evaluate=True)
```

dove `function` è un oggetto SymPy che rappresenta la funzione, `variable` un altro oggetto Sympy che rappresenta la variabile rispetto a cui derivare e `order` un intero che indica l'ordine della derivata. Infine è inoltre specificato il parametro `evaluate` con valore `True`, che indica alla libreria di semplificare, qualora sia possibile, la funzione derivata calcolata. SymPy è infatti in grado mediante determinati algoritmi di intelligenza artificiale di analizzare un'espressione matematica, verificare se semplificabile ed eventualmente scegliere il miglior metodo ed applicarlo per ridurla ai minimi termini.
Ogni microservizio deve avere la sua funzione Python, che con l'utilizzo di SymPy risolve il determinato quesito matematico su determinati parametri ricevuti dall'esterno.

4.1.2 AWS Lambda



AWS Lambda è un servizio fornito da **AWS (Amazon Web Services)**, un'azienda statiunitense di proprietà del gruppo Amazon che mette a disposizione una vasta gamma di **servizi di cloud computing**, tra i quali appunto questo.
Aws Lambda è un **servizio cloud di elaborazione** che permette al programmatore di creare delle funzioni dette **Lambda** che vengono eseguite sui server Amazon secondo determinate **configurazioni**. Le funzioni possono essere scritte in diversi linguaggi di programmazione, tra questi Python.
Le due caratteristiche principali di AWS Lambda sono:

- La possibilità di eseguire le funzioni Lambda senza dover effettuare nessun tipo di provisioning o gestione dei server, creare una logica di dimensionamento delle risorse allocate per l'esecuzione, gestire gli eventi o i runtime; è infatti definito come **serverless**, che sta ad indicare la non necessità del programmatore di preoccuparsi degli aspetti legati al server.

- La sua natura **event driven**: infatti le funzioni caricate dal programmatore sono eseguite quando triggerate da degli eventi, come ad esempio, e come nel mio caso, da una richiesta HTTP con metodo GET ad un determinato endpoint.

Per ogni funzione è possibile andare a configurare determinati parametri; quelli di mio interesse per il progetto sono:

- La **regione**, che indica la zona geografica in cui risiedono i server che verranno utilizzati per l'esecuzione.

- La **memoria allocata** ad ogni esecuzione.

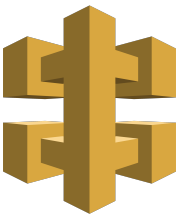
- Un **tempo di timeout**, oltre al quale l'esecuzione della funzione viene interrotta se raggiunto.

AWS Lambda è un servizio a pagamento, ma è possibile utilizzare gratuitamente un **piano base** per la durata di un anno, che ho utilizzato per questo progetto. Esso concede 1.000.000 di richieste mensili complessive alle funzioni attive e 400.000 secondi di esecuzione, ancora mensili e complessivi tra tutte le funzioni esposte. Una volta terminato il periodo di un anno per non bloccare il funzionamento dell'API per il calcolo matematico sarà necessario pagare delle tariffe, ma i prezzi richiesti da Amazon sono assolutamente sostenibili: si parla infatti di 0,23 USD ogni milione

di richieste e 0,0000195172 USD per 1 secondo di esecuzione con 1GB di memoria allocata (quindi la metà con 512 MB allocati, un quarto con 256MB e così via).

Dev'essere quindi creata una funzione Lambda per ogni microservizio, che conterrà ovviamente la funzione Python che realizza il servizio. Ognuna di queste funzioni Lambda dev'essere configurata con la regione **eu-south-1**, i cui server risiedono a **Milano** in Italia, la zona più sensata in quanto l'applicazione è attualmente pensata per essere distribuita in italiano; la memoria di esecuzione dev'essere impostata ad un certo valore che sia sufficiente rispetto alla non indifferente quantità richiesta da SymPy per alcune complesse funzioni, ad esempio 256MB; e infine il tempo di timeout dev'essere configurato ad esempio a 30 secondi, in modo che se l'esecuzione dovesse finire in qualche loop infinito o in qualsiasi altro problema che non la termini, verrà comunque stoppata in modo da non addebitare inutili costi di elaborazione.

4.1.3 AWS Gateway



A questo punto le funzioni Lambda che realizzano i vari microservizi sono pronte per l'esecuzione, bisogna solo renderle **raggiungbili dall'esterno**. Per farlo ho scelto di utilizzare un altro servizio Amazon, **AWS Gateway**, facile da connettere ad AWS Lambda, che permette la creazione di API di varie tipologie, RESTful tra queste.
Posso quindi creare un'API RESTful contenente una route per ogni microservice (indicate nella documentazione), che riceverà le **richieste HTTP** di tipo **GET** in entrata, richiamerà per ognuna di queste la relativa funzione Lambda e risponderà al richiedente con il risultato di quest'ultima. In ogni richiesta GET devono essere specificati i parametri necessari alla funzione Lambda con una **query-string**. Riprendendo l'esempio del microservizio dedicato al calcolo della derivata, una richiesta GET sarà del tipo:

```
https://URL-AWS-Gateway/API/derivative?fun=3*x^2&var=x&n=1&latex=true
```

Dove:

- `URL-AWS-Gateway` indica l'**indirizzo del server Amazon** che espone l'API creata, fornito da AWS Gateway una volta eseguito il deploy dell'ultima.
- `/API/derivative` indica la **route** associata alla funzione Lambda per il calcolo delle derivate. Ogni route creata per essere utilizzata dev'essere necessariamente nel suo path preceduta da /API, come si nota dall'esempio, per motivi legati ad AWS Gateway.

- `fun`, **parametro della query-string**, è una stringa che indica la funzione da derivare. É importante ricordare che eventuali caretteri speciali, come ad esempio l'apice presente nell'esempio, per effettuare la richiesta al server devono essere convertiti con un URL Encoder.
- `var`, **parametro della query-string**, è una stringa che indica la variabile rispetto alla quale derivare.
- `n`, **parametro della query-string**, è un intero che indica l'orinde della derivata da calcolare.
- `latex`, **parametro della query-string**, è un valore booleano che indica se il formato della risposta sarà il **LaTeX** (formato che descriverò più avanti) o no (l'aternativa è una **stringa human-readable**).

Ovviamente ogni route avrà i suoi specifici parametri, indicati per ogni servizio nella documentazione, ma la struttura di base della richiesta rimane la stessa.
La risposta ad una richiesta sarà specificata in formato **JSON** nel body del messaggio HTTP di risposta, con determinate chiavi specificate nella documentazione. É inoltre prevista una **gestione degli errori**, principalmente quelli di input, segnalati nel body della riposta HTTP in caso di eccezione. Ogni errore, oltre ad una stringa descrittiva, ha un suo **ID** che permette al programmatore che utilizza le API una più semplice gestione dei diversi errori. Gli errori previsti per ogni servizio e i loro relativi ID sono anch'essi indicati nella documentazione.

A questo punto i microservizi per il calcolo matematico sono pronti per essere utilizzati dal client Angular. Illustrerò prima però l'altra parte del lato server del progetto, quella relativa al server Spring Boot e al database, per poi passare al lato client nel capitolo successivo.

4.2 API per l'interazione col database

Per quanto riguarda questa sezione ho progettato un'**API** per interagire con un **database relazionale** in cui memorizzare principalmente i dati relativi all'autenticazione degli utenti che utilizzeranno l'applicazione e ai capitoli teorici contenuti nella sezione di teoria.

4.2.1 Database

Analisi della realtà

Ho scelto realizzare un database di tipo relazionale per rappresentare le informazioni utili ai fini dell'applicazione. In primo luogo ho analizzato la realtà che dev'essere rappresentata, composta da:

- Una serie di **utenti**, ciascuno dei quali avrà un certo *username*, una *password* per l'accesso e una certa *email*. Ogni utente può registrarsi all'applicazione inserendo tutti i suoi dati richiesti, e il suo account verrà attivato solo dopo la sua interazione con una email di conferma.

- Una serie di **piani tariffari**, ognuno dei quali ad un certo *prezzo mensile* garantisce all'utente l'accesso a determinati **servizi di calcolo matematico** e a determinati argomenti di teoria. Ogni piano, come ogni servizio di calcolo, ha una breve *descrizione*.

- La lista di tutti gli **argomenti di teoria**, ognuno dei quali ha una sua breve *descrizione* e comprende dei capitoli scritti e/o a sua volta altri sottoargomenti.

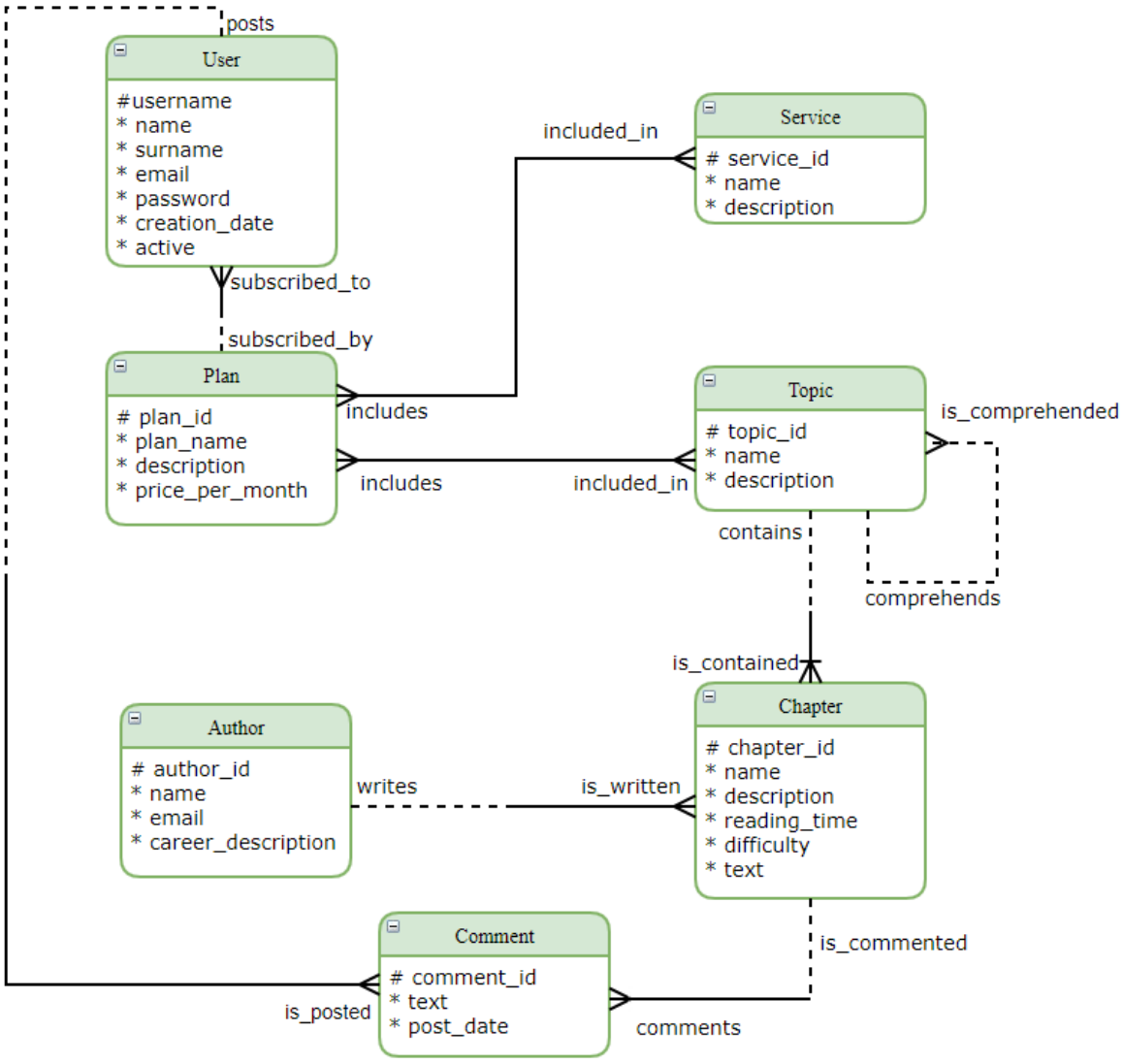
- La lista di tutti i vari **capitoli**, contenuti negli argomenti, ognuno dei quali presenta, oltre che ovviamente al suo contenuto di testo, una breve *descrizione*, un certo *tempo di lettura* (in minuti) e un *grado di difficoltà*. Il *contenuto di testo* di un capitolo sarà un documento scritto in formato **LaTeX**, un linguaggio di markup per la stesura di testi, che facilita notevolmente l'inserimento di formule matematiche immancabili in questi capitoli.

- Inoltre di ogni capitolo si conosce l'**autore**, caratterizzato da un *indirizzo email* e da una breve *descrizione della sua carriera*.

- Infine ogni utente ha la possibilità di lasciare uno o più **commenti** per ogni capitolo di teoria.

Progettazione concettuale

Sono poi passato alla progettazione concettuale del database, utilizzando il **modello E-R** (**Entità-Relazione**):

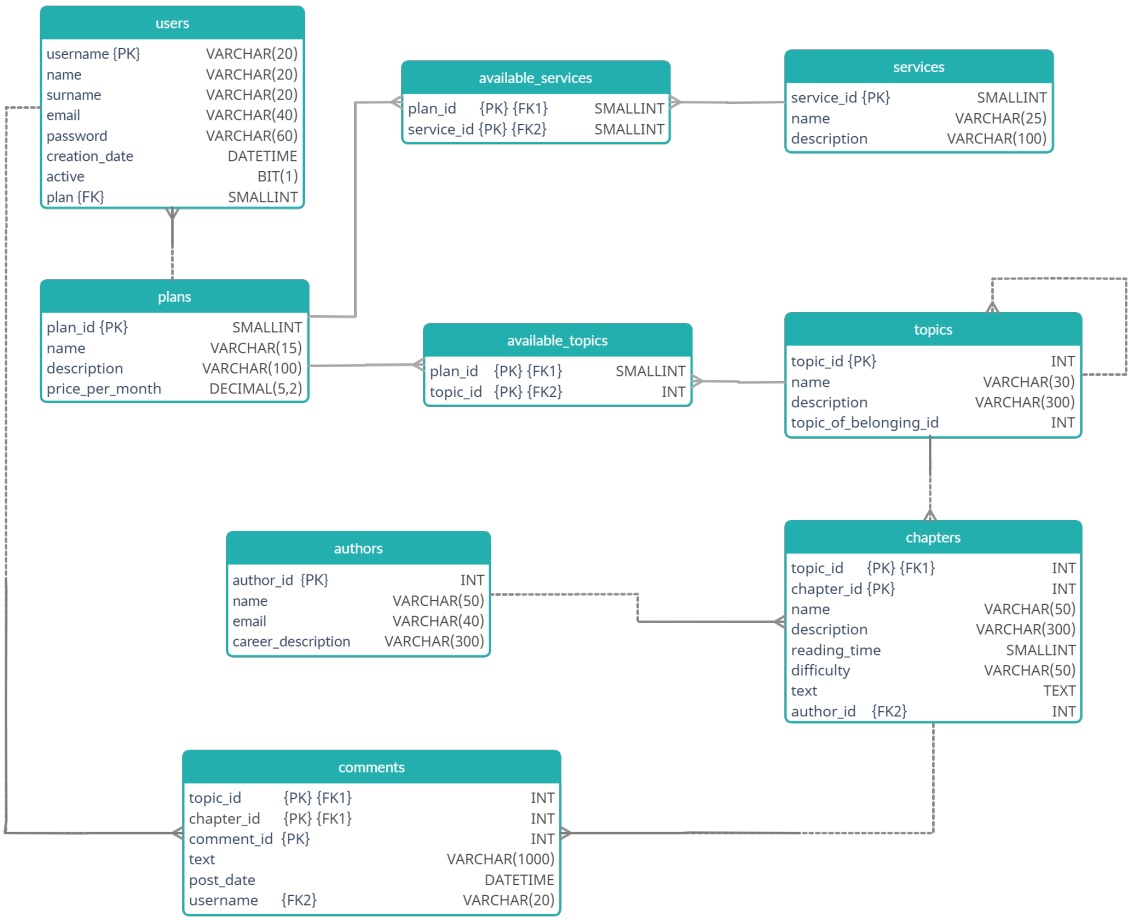


Guardando lo schema, si potrebbe avere ad esempio un utente, Mario Rossi, con username `mario.rossi` che ha creato ed attivato il suo account con l'ipotetico piano `PRO`. Questo piano, per quanto riguarda la sezione di calcolo matematico, gli concede l'accesso a tutti i servizi di calcolo previsti dall'applicazione, attualmente

dunque i risolutori di equazioni, disequazioni, limiti, derivate, integrali ed equazioni differenziali. Per quanto riguarda la sezione di teoria invece potrà visualizzare tutta una serie di argomenti, tra i quali ad esempio, quello riguardante i numeri complessi, che conterrà, oltre che ad un ipotetico sottoargomento dedicato alla storia dei numeri complessi, una serie di capitoli appartenenti a questo ambito. Ognuno di questi capitoli avrà un riferimento ad un suo autore, di cui Mario Rossi potrà visualizzare i dati, ed eventualmente una serie di commenti lasciati da altri utenti.

Progettazione logica

A questo punto ho proceduto con il **mapping** dal modello E-R a quello **relazionale**, risolvendo tutte le associazioni del precedente diagramma in questo modo:



Vorrei soffermarmi su alcune scelte per i tipi di determinati attributi: per quanto riguarda l'attributo `password` della relazione `users`, ho scelto un `VARCHAR` di dimensione 60 in quanto le password saranno salvate in formato criptato con un Encoder che codifica le password in stringhe da 60 caratteri. Agli ID delle relazioni `users`, `plans`, `services` ho assegnato il formato `SMALLINT`, che prevede un valore intero massimo di 32.767, numero sicuramente sufficiente per tutte e tre le relazioni. Tutti gli altri ID verranno memorizzati in formato `INT`, che prevede un massimo di 2.147.483.647, dato che c'è la remota possibilità che per queste altre relazioni il formato `SMALLINT` non sia sufficiente.

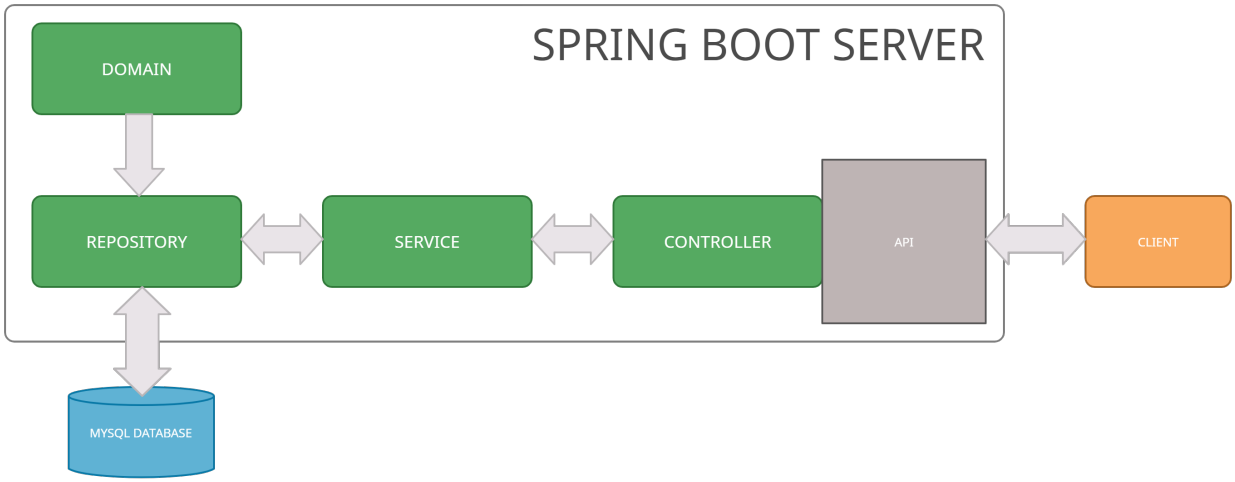
Il database è a questo punto dunque pronto per essere implementato. Per farlo però non ho utilizzato le classiche istruzioni SQL, ma mi sono servito di una tecnologia di approccio moderno molto efficace, che adesso andrò ad illustrare.

4.2.2 Server Spring Boot

Spring Boot è un **framework open source** per lo sviluppo di **applicazioni web** nel linguaggio di programmazione **Java**, estensione di **Spring**. A differenza del secondo, Spring Boot permette la creazione di un'applicazione Java **a sè stante**, senza quindi la necessità di un server al quale appoggiarsi per essere eseguita, e con una più semplice configurazione. Esso comprende degli importanti strumenti per far interagire Java con un **DBMS**, nel mio caso **MySQL**, ma non solo. Infatti permette anche di creare in modo semplice e ordinato delle **route REST** che le applicazioni esterne possono utilizzare per comunicare con quella Spring Boot.
Questo framework è dunque quello che fa al caso mio, in quanto permette la creazione di un server raggiungibile con un'API REST in grado di interagire con un database. Ovviamente in questo caso, a differenza della sezione server per il calcolo matematico, l'API rimarrà privata, in quanto servirà esclusivamente al client del progetto, per autenticare i vari utenti, attivar loro i servizi di calcolo relativi al loro piano e render loro disponibili i vari argomenti teorici memorizzati nel database.

Architettura del Server

L'architettura alla base del server Spring Boot è la seguente:



Ognuno dei 4 elementi appartenenti al server ha la sua specifica funzione:

- La parte di **domain** permette di definire tutte le **relazioni** presenti nel database e i **vincoli di integrità referenziali** che le legano, che in questo modo avranno la loro rappresentazione in **oggetti Java**.
- La sezione di **repository** si preoccupa invece di fare da tramite tra la rappresentazione delle varie relazioni in oggetti Java e quella all'interno del database. Grazie ad essa si può infatti caricare con una semplice istruzione una tupla di una qualsiasi relazione in un oggetto Java, o viceversa. Questo approccio è detto **ORM** (**Object-Relational Mapping**), che, semplificando, è una tecnica di programmazione che permette di richiamare, aggiornare ed eliminare dati da un database all'interno di programmi object-oriented.
- La parte di **service**, che intergisce col database grazie alla repository, si occupa di gestire invece tutta la **business logic** del server, come succede per il Model nel classico pattern Model-View-Controller.
- Nella sezione di **controller**, infine, vengono dichiarate tutte le **route** che le applicazioni esterne utilizzeranno per interagire col database. Essa si occupa infatti di ricevere i **messaggi HTTP** dal client ed inoltrarli al service, che li processa per poi restituire un esito dell'elaborazione della richiesta al controller, il quale risponderà a sua volta al client con un messaggio HTTP.

Creazione del database ed interazione

Un punto forte di Spring Boot è la semplicità con la quale si può creare il database ed interagirvi.
Per farlo è necessario includere nel server il modulo `spring-boot-starter-data-jpa`, che si basa su **JPA**, un'API Java che si occupa della **persistenza dei dati** (la capacità di quest'ultimi di sopravvivere all'esecuzione del programma che li ha creati o modificati) di un DBMS relazionale in un'applicazione Java. Grazie a questo modulo, sarà sufficiente andare a definire nel domain tutte le vari relazioni rappresentate nel database e le associazioni tra di esse, e il framework, in fase di bootstrap del server, sarà in grado se necessario di creare il database, senza il bisogno di scrivere nessuna istruzione SQL.

Ogni entità nel domain dev'essere rappresentata tramite una classe Java, con le relative **annotation** che indicano al framework cosa rappresentano i vari componenti della classe:

```
@Entity
@Table(name="users")
public class User{
    @Id
    @Column(name="username",length=20)
    String userName;

    @Column(name="name",length=20,nullable=false)
    String name;

    ...
```

Ad esempio in questo caso la classe `User` sarà interpretata dal framwework come una relazione, che avrà la sua rappresentazione nel database tramite la tabella `users`. L'attributo `userName` della classe sarà interpretato come colonna della tabella `users`, che è inoltre chiave primaria, come indica l'annotation `@Id`.

Inoltre all'interno di queste classi è necessario specificare le associazioni che intercorrono tra le varie relazioni, ancora con l'utilizzo delle specifiche annotation. Ad esempio, per implementare l'associazione *many-to-one* tra la tabella `users` e la tabella `plans` (ogni utente ha un piano, un piano può essere attivo per uno o più utenti) sarà sufficiente aggiungere la relativa annotation sull'attributo che rappresenta la colonna `plan_id` della tabella `users`, chiave esterna verso la tabella

`plans`:

```
    ...

    @ManyToOne(optional = false)
    @JoinColumn(name="plan_id",referencedColumnName = "plan_id")
    Plan plan;
}
```

La classe `Plan` è un'altra classe del dominio, che rappresenta la relazione `plans`.

Per l'interazione del service con il database, come detto prima, è necessario ora andare a definire per ogni relazione, e quindi elemento del domain, con la quale si vuole interagire, una repository. Per farlo è sufficiente creare una **Java interface**, segnata con l'apposita annotation, che estende l'interface `JpaRepository`:

```
@Repository
public interface UserRepository extends JpaRepository<User,String> {}
```

I tipi specificati tra parentesi angolare indicano rispettivamente il tipo del domain da utilizzare per rappresentare la relazione e il tipo della sua chiave primaria. A questo punto nel service, dichiarando un oggetto di tipo `UserRepository` che sarà automaticamente instanziato dal framework, è possibile andare ad interagire coi dati del database, facendo per esempio delle ricerche per ID, modificando alcune colonne di una tupla, o eliminandone.
C'è inoltre la possibità di definire dei metodi, all'interno delle repository, che restituiscano il risultato di una **query SQL nativa** specificata. Ad esempio nella repository che si occupa della tabella `topics` ho inserito un metodo che restituisce il risultato di una query che, dato l'ID di un certo piano, trova tutti i topic che esso rende accessibili:

```
@Repository
public interface TopicRepository extends JpaRepository<Topic,Integer> {
    @Query(value = "
    SELECT t.topic_id, t.name, t.description, t.sub_topic
    FROM topics t JOIN available_topics a ON t.topic_id = a.topic_id
    WHERE a.plan_id = :plan_id
    ", nativeQuery = true)
    List<Topic> getAvailableTopics(@Param("plan_id") Integer planID);
}
```

Esposizione con l'API

Ora che la creazione del database e l'interazione con esso sono garantite, si procede con la scrittura delle classi che fungono da service, utilizzate poi dalla sezione controller per elaborare le richieste HTTP ricevute, e da controller. É comodo avere per ogni elemento del dominio una classe di service e una di controller. In questo modo ogni classe del controller espone le route relative alla sua entità, ad esempio per la tabella `topics` si avrà la seguente classe:

```
@RestController
@RequestMapping("/api/topics")
@CrossOrigin(origins = "*")
public class TopicController {
    @Autowired
    TopicService topicService;

    @GetMapping("/available")
    public ResponseEntity<?> getAvailable(){
        List<Topic> availableServices = topicService.getAvailable(...);
        return new ResponseEntity<>(availableServices, HttpStatus.OK);
    }

    ...
```

La route per l'ottenimento della lista di topic disponibili per l'utente richiedente sarà quindi raggiungibile all'endpoint `http://URL_SERVER/api/topics/available`, inviando un messaggio HTTP con metodo GET.
In questo modo posso andare a definire tutte le varie route necessarie al funzionamento dell'applicazione, esponendo così un'API utilizzabile dal client del progetto.

Sicurezza e autenticazione

Come detto in precedenza quest'API non dev'essere pubblica, e quindi raggiungibile solo una volta effettuata un'**autenticazione**. Altrimenti chiunque potrebbe, per esempio, farsi inviare dal server con una semplice richiesta HTTP qualsiasi argomento di teoria disponibile nel database. Per gestire questo aspetto ho utilizzato **Spring Security**, un **framework** incorporabile nelle applicazioni Spring volto a rendere semplice e allo stesso tempo altamente personalizzabile l'implementazione di un sistema di autenticazione.

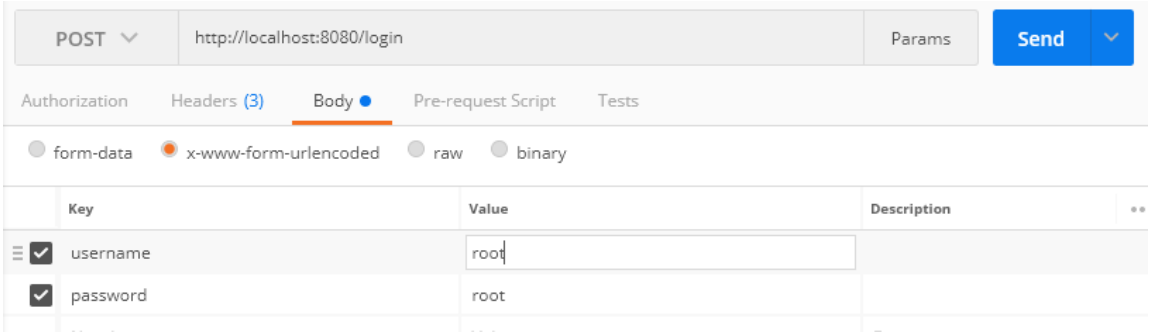
Per rendere disponibili i servizi di Spring Security al server basta semplicemente aggiungere il modulo `spring-boot-starter-security`, e andare a configurarlo con un'apposita classe Java. Importante è l'impostazione della tabella del database dalla quale il server andrà a prelevare i dati sugli utenti per autenticarli:

```
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Autowired
    private DataSource dataSource;

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
        Exception{
        auth.jdbcAuthentication().dataSource(dataSource)
                .usersByUsernameQuery("SELECT username,password,active FROM
                    users where username=?")
                .authoritiesByUsernameQuery("SELECT username,plan_id as
                    authority FROM users where username=?")
                .passwordEncoder(getPasswordEncoder());
    }

    ...
```

Grazie a `JDBC`, un **driver** che consente ad un'applicazione Java di interagire con un database, si possono specificare le query SQL che devono essere eseguite per recuperare i dati sugli utenti utili all'autenticazione. Inoltre come si può notare dal codice in questo metodo di configurazione si può andare a specificare un **password encoder**: le password nel database sono previste per non essere memorizzate in formato di testo chiaro, ma **criptato**. Andando a specificare un encoder in questo modo il server, quando andrà a verificare se una passsword in entrata è corretta, saprà in che modo confrontarla con quella criptata presente nel database. La **funzione di hashing** da me scelta per l'encoding delle password è **Bcrypt**.
Una volta attivato il modulo di autenticazione il server bloccherà ogni route indicata nella configurazione alle richieste non contenenti i giusti dati di autenticazione. Per autenticarsi è necessario innanzitutto un **messaggio HTTP** di tipo **POST** di **login**, all'apposita route, che avvia la **sessione** del client nel server, e successivamente, ad ogni richiesta, il richiedente dovrà aggiungere negli **header** i suoi **dati di autenticazione**. Il messaggio di POST per il login sarà, per esempio:



Una volta eseguito il login il client, per ricevere i dati dalle altre route, dovrà specificare nell'header `Authorization` delle richieste le sue credenziali, nel formato previsto da `Http Basic`, metodo di autenticazione da me indicato nella configurazione di Spring Security.

Un ultimo aspetto legato alla sicurezza è la **registrazione** degli utenti: infatti la route per la registrazione, raggiungibile ovviamente senza il bisogno di essere autenticati, prevede che al richiesta contenga un body che indica tutti i dati relativi all'utente che si sta andando a registrare, password inclusa. Quest'ultima però non arriva al server criptata, dev'essere infatti elaborata prima di essere inserita nel database. Nel mio caso di questo si occupa il service degli utenti, con un metodo richiamato dal controller per la registrazione, il quale con un'istruzione cripta la password con l'encoder di cui parlavo prima:

```
user.setPassword(BCrypt.hashpw(user.getPassword(), BCrypt.gensalt()));
```

Una volta criptata la password l'utente è pronto per essere inserito nel database e i suoi dati saranno utilizzabili per l'autenticazione (dopo aver confermato la registrazione con la mail di conferma).

A questo punto tutto il lato server necessario al progetto è disponibile, raggiungibile con delle API Rest, e pronto per essere utilizzato dal client. Per quanto riguarda la **sicurezza della trasmissione**, relativamente alla parte di microservizi, anche se non è molto importante in questo caso, non ci sono problemi, in quanto l'API esposte da AWS Gateway prevedono il protocollo **HTTPS**. Il server Spring Boot invece, per un'iniziale **fase di beta** dell'applicazione, può essere pubblicato su una qualche **piattaforma di deploy**, che è importante garantisca HTTPS. Un esempio sicuramente valido è heroku, che prevede dei piani gratuiti adatti ad una prima pubblicazione dell'applicazione.

5

Client

Andrò ora ad illustrare il lato client dell'applicazione, che consiste in un'intefaccia web realizzata in **Angular** che utilizza le API realizzate per il suo funzionamento.

5.1 Angular



Angular è un **framework** sviluppato da **google** (principalmente) per lo sviluppo di applicazioni web. Esso utilizza il linguaggio di programmazione **TypeScript**, e fornisce una grande vastità di **librerie** utili allo sviluppo.
Ho quindi scelto di utilizzare questo framework, che permette in modo semplice lo sviluppo di applicazioni web **scalabili** e, inoltre, **graficamente piacevoli**.

5.1.1 Component e Service

Un dei concetti alla base di questo framework sono i **Component**, dei blocchi che insieme compongono l'applicazione. Ogni component ha una parte **HTML** e **CSS** per quanto rigurada la visualizzazione, e una parte **TypeScript** che si occupa della logica e quindi dei "comportamenti" che deve assumere il component. Un altro concetto importante è quello di **Service**, ovvero una classe TypeScript che espone dei metodi utili alla logica dell'applicazione e utilizzabili dai vari component, ad esempio come nel mio caso funzioni che eseguono richieste al server Spring Boot.
Ho creato un component per ogni pagina dell'applicazione, il che semplifica lo sviluppo

in quanto suddivide in unità indipendenti il client, più semplici da gestire. I component che ho creato sono:

- Uno di **login**, contenente un form per l'accesso.
- Uno di **registrazione**, contenente un form per la registrazione in caso l'utente non abbia già un account.
- Uno per la **home**, dalla quale è possibile accedere ai vari servizi di calcolo matematico e alla sezione di teoria.
- Un component per ogni **strumento di calcolo**, che, dato un certo input utente, fa una richiesta allo specifico microservizio Lambda e mostra a video il risultato.
- Il principale della **sezione di teoria**, che permette di visualizzare la lista di argomenti disponibili sul server al tipo di utente loggato, ed eventualmente aprirne uno.
- Uno per i **capitoli di teoria**, che visualizza la lista di capitoli contenuti in un argomento ed eventualmente i vari sotto argomenti, e ne permette l'apertura.
- Uno per la visualizzazione di un singolo **capitolo**.
- Uno di **errore**, caricato nel caso in cui l'utente cerchi di raggiungere un URL dell'applicazione inesistente.

Tutti i component, esclusi i primi due, non sono raggiungibili prima di aver eseguito il login (o eventualmente la registrazione, che effettua in automatico il login).

I component, per garantire i vari servizi, utilizzano, in base alle loro esigenze, determinati service. I service utili all'applicazione da me creati sono:

- un **LambdaService**, in grado di inviare richieste HTTP ai vari microservizi Lambda.
- un **DBService**, utile per la comunicazione HTTP col server Spring Boot.
- un **AuthService**, che si occupa dell'autenticazione dell'utente e della gestione della sua sessione.
- un **RouterService**, utile per la gestione della navigazione tra i diversi component (quindi tra le diverse pagine).

5.1.2 Directive

Un concetto fondamentale di Angular, uno dei suoi grandi punti di forza, è quello di **directive**. Una directive, semplificando, è una proprietà attribuibile ad un tag HTML che assegna ad esso un determinato comportamento. É incluso in Angular un insieme di directive built-in, ma è anche possibile dichiararne delle custom. All'interno di questa applicazione ho utilizzato principalmente due directive, già incluse in Angular: **ngFor** e **ngIf**.

ngFor

Questa directive, se assegnata ad un certo tag HTML, lo ripete per ogni elemento di una lista. Ad esempio, per il caricamento dei vari argomenti di teoria disponibili, è sufficiente scrivere un singolo elemento HTML, che verrà poi ripetuto per ogni argomento:

```
<mat-grid-list cols="1" rowHeight="110px">
  <mat-grid-tile *ngFor="let topic of topics">
    ...
  </mat-grid-tile>
</mat-grid-list>
```

Il valore della variabile `topic` cambia per ogni elemento HTML, e può essere utilizzato, con la tecnica di **Data Binding** che discuterò tra poco, per compilare i vari campi dell'elemento.

ngIf

Questa directive, se assegnata ad un certo tag HTML, determina in base ad un valore booleano indicato se esso sarà visualizzato o meno. In ogni pagina degli strumenti per il calcolo matematico, è prevista una label di errore in caso di problemi ad esempio con l'input dell'utente. Questa label dev'essere visualizzata solo in caso ci sia per l'appunto un errore, il che è realizzabile in modo molto semplice grazie alla directive in questione:

```
<div id="math-error" *ngIf="error">
  ...
</div>
```

La variabile `error`, attributo della classe TypeScript legata al singolo component, determina col suo valore booleano se la label è visualizzata o no.

5.1.3 Data Binding

Un altro dei concetti fondamentali di Angular, e anch'esso uno dei suoi più grandi punti di forza, è quello del Data Binding. Grazie ad esso è possibile **scambiare dati** in modo semplice tra la classe TypeScript di un component ed il suo **DOM** (**Document Object Model**), ovvero la rappresentazione in oggetti JavaScript dei vari tag HTML contenuti all'interno del component. Per utilizzare questa tecnica è sufficiente inserire determinati comandi, se così si possono chiamare, all'interno dei tag HTML definiti nel file HTML del component. Esistono 4 diverse forme di Data Binding ognuna delle quali ha la sua specifica situazione di utilizzo.

Interpolation

Grazie all'interpolation è possibile assegnare in modo dinamico il valore di una **variabile TypeScript** al **contenuto HTML** di un determinato tag. Ad esempio, riprendendo in esame la compilazione della lista dei vari argomenti di teoria, per ognuno di essi sarà necessario visualizzare titolo e descrizione, il che è realizzabile semplicemente inserendo nei tag HTML le seguenti stringhe:

```
<mat-grid-list cols="1" rowHeight="110px">
  <mat-grid-tile *ngFor="let topic of topics">
    <mat-card>
      <mat-card-title>
        <strong>
          {{ topic.name }}
        </strong>
      </mat-card-title>
      <mat-card-content>
        {{ topic.description }}
      </mat-card-content>
    </mat-card>
  </mat-grid-tile>
</mat-grid-list>
```

La variabile `topic`, fornita dalla directive `ngFor`, contiene dei campi che possono essere assegnati come valori dei tag HTML grazie appunto all'interpolation. Ed ogni volta che il valore delle variabili TypeScript indicate cambia, esso viene automaticamente aggiornato anche nel DOM (anche se nell'esempio appena illustrato non può succedere). É inoltre possibile indicare delle **funzioni** invece che delle variabili, il cui valore di return sarà assegnato al contenuto del tag HTML.

Property Binding

Grazie al Property Binding è possibile, con una modalità simile a quella dell'Interpolation, assegnare agli **attributi** dei tag HTML valori contenuti in **variabili TypeScript** o restituiti da determinate **funzioni**. Ad esempio nel form di registrazione, è presente un elemento `mat-option` per la scelta del piano utente che si vuole utilizzare, le cui opzioni, i vari piani, devono essere caricate dinamicamente in base ai dati presenti nel database. Ogni opzione deve avere come valore dell'attributo `value` l'ID del piano, il che è realizzabile in modo molto semplice grazie al Property Binding combinato ancora una volta alla directive `ngFor`:

```
<mat-option *ngFor="let plan of plans" [value]="plan.planID">
    {{ plan.planName }}
</mat-option>
```

Event Binding

L'Event Binding permette di gestire in modo semplice gli **eventi**, come il click di un bottone o la pressione di un determinato tasto della tastiera. Per esempio, ancora riprendendo la lista di argomenti, per gestire il click su uno degli argomenti è sufficiente lavorare sul file HTML:

```
<mat-grid-list cols="1" rowHeight="110px">
  <mat-grid-tile *ngFor="let topic of topics"
      (click)="openTopic(topic.id)">
    <mat-card>
```

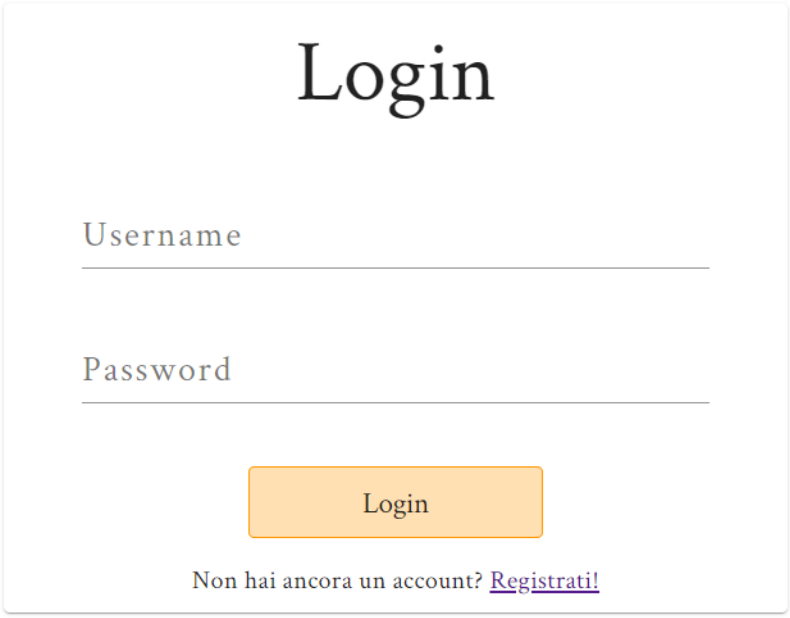
In questo modo, ogni volta che l'utente cliccherà sulla `mat-grid-tile` rappresentante l'argomento, verrà richiamato il metodo `openTopic(topicID)` con l'ID specificato grazie ai dati ricevuti dalla directive `ngFor`.

L'ultimo tipo di binding è il **Two-way Data Binding**, che non ho però utilizzato in modo significativo e che non andrò dunque a discutere.

5.2 Angular Material

Per quanto riguarda la **parte HTML** dell'applicazione ho scelto di utilizzare Angular Material, un **toolkit** che fornisce una serie di componenti HTML (come quelli visti

negli esempi di codice precedenti) facili da gestire ed inserire nei vari component. Esso semplifica molto lo sviluppo della **parte strutturale** e **grafica** delle pagine, dando inoltre una certa **estetica**, anch'essa facile da gestire.
Ad esempio, nella pagina di login, ho utilizzato vari componenti forniti da Angular Material, come il bottone di login e le textbox per l'inserimento di nome utente e password, e con semplicità ho raggiunto un risultato di una certa estetica:



5.3 Login e Autenticazione

Per quanto riguarda il login, come precedentemente spiegato, il server si aspetta un **messaggio HTTP** di tipo **POST** alla **route di login** contente le **credenziali di accesso**, che se accettate renderanno disponibili le **altre route**. Dalla pagina di login sarà dunque possibile inserire nel **form** nome utente e password, e premendo il pulsante di login verrà inviato il messaggio HTTP. Nel caso il server risponda con esito positivo l'applicazione si sposterà sulla pagina **Home**, altrimenti verrà visualizzato un messaggio di errore.
Per quanto riguarda le richieste alle altre route Spring Boot, come precedentemente spiegato, gli **header** dovranno contenere il campo di **autenticazione**. Di questo, come del messaggio di login, si preoccupa l'*AuthService*. Infatti per la compilazione

delle options (che contengono i vari header) per le richieste HTTP una volta effettuato il login, il service in questione espone il seguente metodo:

```
static getOptions(): any {
    return {
        headers: new HttpHeaders().append('Authorization',
        'Basic ' +
        btoa(AuthService.user.username + ':' + AuthService.user.password))
    };
}
```

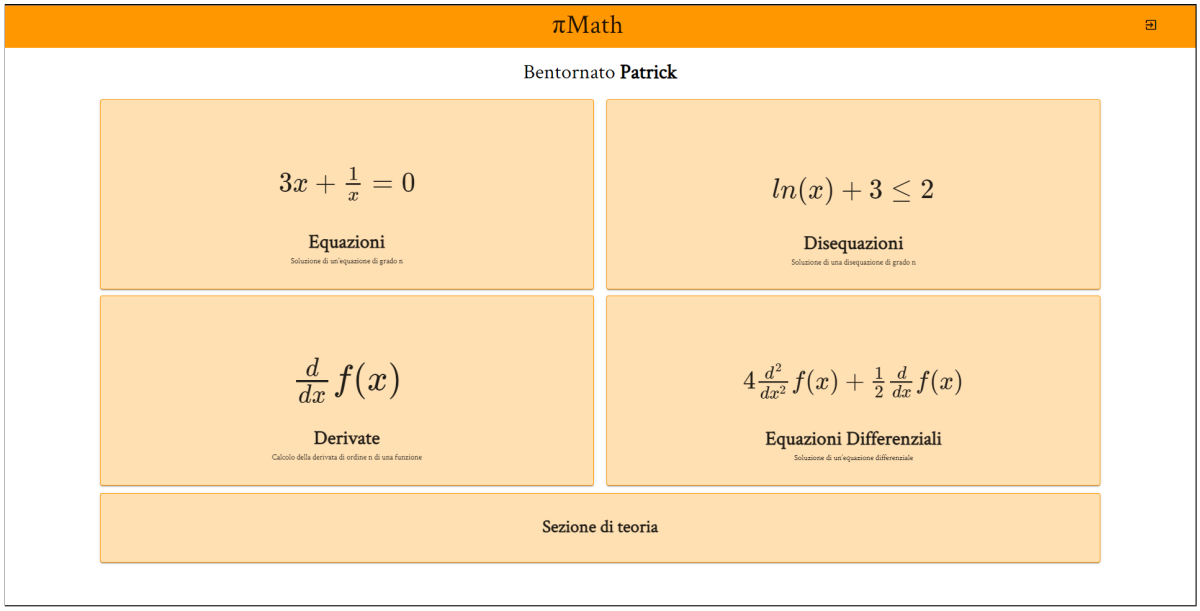
Un'altra funzione svolta dall'*AuthService* è quella di **logout**: è esposto infatti un metodo che, se richiamato da un component ad esempio in seguito al click di un apposito bottone, invia un messaggio alla route Spring Boot di logout:

```
logout(): Observable<any> {
  AuthService.loggedIn = false;
  AuthService.activeServices = [];
  return this.http.get(DBService.URL + 'logout', AuthService.getOptions());
}
```

In seguito al logout l'utente è reindirizzato alla pagina di login.

5.4 Sezione di calcolo matematico

Una volta raggiunta la home, verrà visualizzata una serie di pulsanti:



Uno per ogni strumento di calcolo matematico e uno per la navigazione verso la sezione di teoria. Al caricamento del component esso recupera dal server Spring Boot, grazie al *DBService*, la lista degli strumenti di calcolo matematico disponibili grazie al piano dell'utente, e carica un pulsante per ognuno di essi.

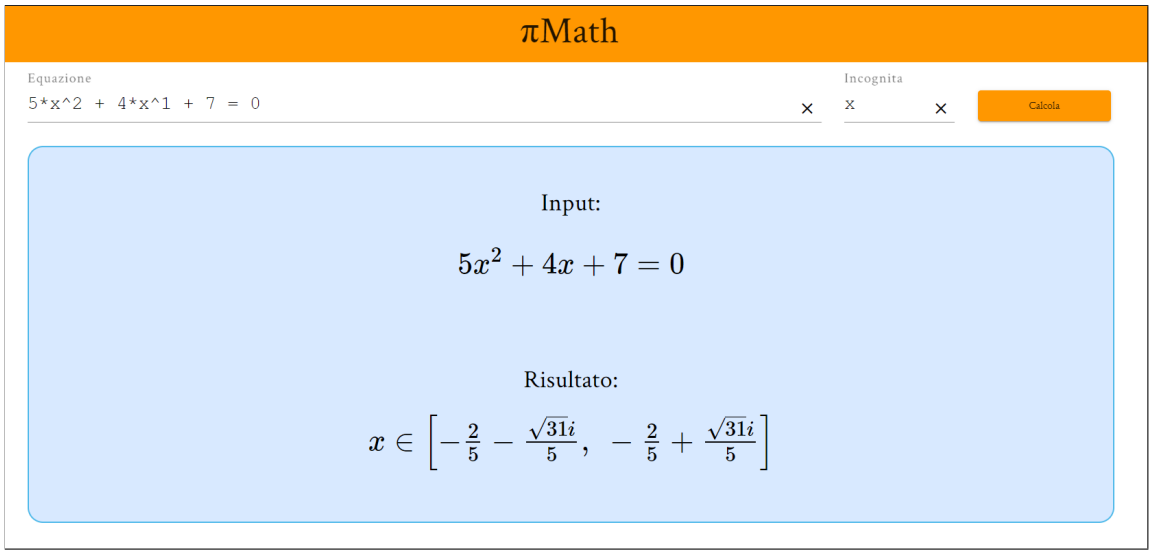
5.4.1 MathJax

In un'applicazione come la mia, come si può oltretutto notare dall'immagine soprastante, è importante la visualizzazione di **espressioni matematiche**. Per quest'aspetto ho utilizzato MathJax, una **libreria JavaScript** per la visualizzazione nel browser di espressioni matematiche. MathJax è in grado di leggere stringhe di determinati formati, come ad esempio **LaTeX** o **MathML**, interpretarle e convertirle in elementi HTML per la visualizzazione nel browser. Nel mio caso ho utilizzato il formato LaTeX, dato che i microservizi Lambda sono in grado di resituire l'output in questo formato. LaTeX è un **linguaggio di markup** per la stesura di documenti, grazie al quale è possibile, tra le molte funzionalità, visualizzare con un'elegante formattazione espressioni matematiche. MathJax è in grado di elaborare solo una piccola sezione degli innumerevoli tag LaTeX, tra questi, però, tutti quelli a me

utili per visualizzare i vari simboli matematici. Ad esempio, inserendo la stringa `$\frac{d}{dx} f{\left(x \right)}$` all'interno del pulsante per l'apertura dello strumento per il calcolo delle derivate, verrà visualizzato il risultato nella precedente figura.

5.4.2 Strumenti di calcolo

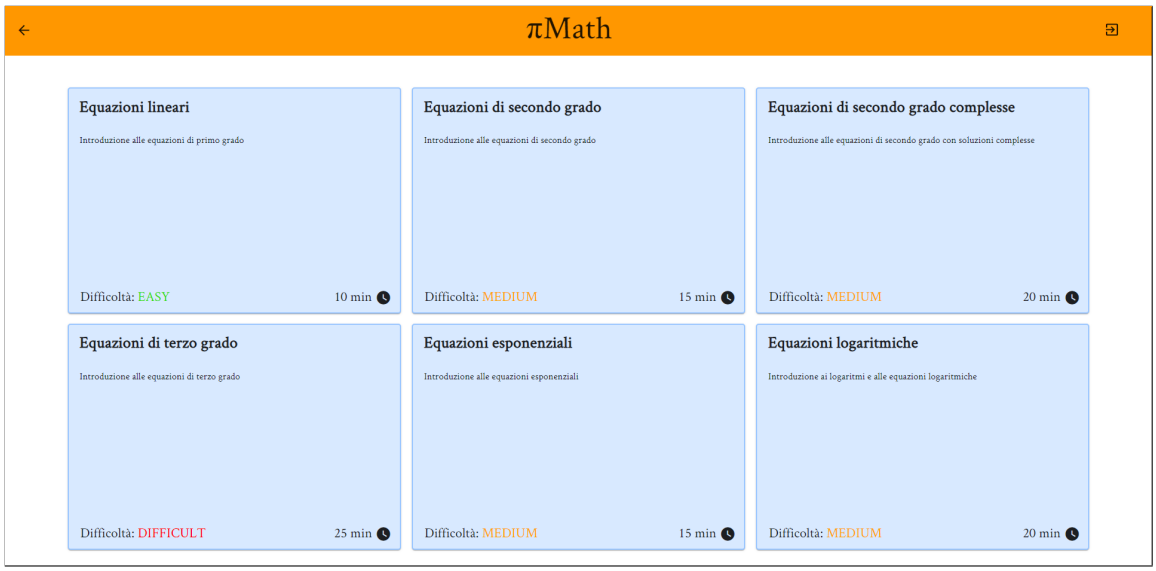
Cliccando su uno dei pulsanti verrà caricato il **component dello specifico servizio**, che metterà a disposizione un **form** per inserire i **dati di input** di un determinato calcolo che si vuole eseguire, e un pulsante per far partire l'elaborazione. Una volta premuto il bottone il component, utilizzando il *LambdaService*, invierà una richiesta allo specifico microservizio e visualizzerà la risposta a video.



Attualmente per l'input di espressioni matematiche è previsto un semplice **formato testo di base**. Questo richiede il rispetto di alcune regole, ad esempio tra un'incognita e il suo coefficiente dev'essere necessariamente inserito un `*`. Un altro sicuro sviluppo futuro è infatti uno strumento per l'inserimento delle espressioni matematiche.
Nel caso in cui la risposta contenga degli errori essi possono essere gestiti grazie ai codici di errore progettati per le funzioni lambda di cui ho precedentemente parlato, cosicchè il programma sia in grado di capire cosa è andato storto e comunicarlo con uno specifico messaggio di errore all'utente.

5.5 Sezione di teoria

Se, nella home, l'utente preme il pulsante per la sezione di teoria, verrà caricato il component contenente la **lista di argomenti disponibili**. Per caricare questi ultimi verrà fatta, tramite il *DBService*, una richiesta al server Spring Boot, che risponderà con la lista degli argomenti disponibili in base al piano dell'utente richiedente. In caso di problemi col recupero dei dati l'utente verrà reindirizzato alla pagina precedente (in questo caso la home) e verrà visualizzato un messaggio di errore, come succede per tutti gli altri component riguardanti la sezione di teoria.
Ogni elemento della lista di argomenti, contenente il **titolo** e la **descrizione** dell'argomento, sarà cliccabile, e, se appunto cliccato, verrà caricato il component per la **visualizzazione dei capitoli** (ed eventuali **sottoargomenti**), anche questa volta ricevuti grazie ad una richiesta del *DBService* al server Spring Boot. Andando ad aprire un ipotetico argomento dedicato alle equazioni, si avrà ad esempio il seguente risultato:



Cliccando su uno dei capitoli verrà caricato il component dedicato e visualizzato il **contenuto del capitolo**.

5.5.1 LaTeX.js

Una volta aperto il component per la **visualizzazione del singolo capitolo**, ancora una volta verrà fatta una richiesta al server Spring Boot, che risponderà con il **contenuto** del capitolo da visualizzare. Come precedentemente detto, i capitoli di

teoria non sono altro che dei brevi **documenti LaTeX**, che quindi necessitano di una certa elaborazione per essere visualizzati. MathJax, essendo in grado di elaborare solo una piccola sezione dei tag LaTeX, non è sufficiente per la visualizzazione di un documento come un capitolo di teoria. Per questo ho utilizzato invece **LaTeX.js**, molto adatto alla mia situazione. LaTeX.js è una **libreria JavaScript** in grado di visualizzare all'interno del browser interi documenti scritti in formato LaTeX, grazie ad uno specifico tag HTML da essa reso disponibile: sarà infatti sufficiente inserire il documento LaTeX all'interno del tag `latex-js`:

```
<latex-js baseURL="https://cdn.jsdelivr.net/npm/latex.js/dist/">
  {{ chapterContent }}
</latex-js>
```

E in questo modo, una volta ricevuto, il capitolo di teoria sarà elaborato e mostrato a video.

A questo punto anche il client è pronto, dunque tutta l'applicazione può essere caricata su dei servizi di deploy, o meglio, come previsto per il futuro, esposta sulla rete privata descritta nella sezione dedicata all'architettura di rete, per poi essere utilizzata dagli utenti tramite il browser.

6

Conclusioni e riflessioni

Concludo con questo progetto un importante periodo della mia vita. Questi tre anni, seguito della mia scoperta della passione per l'informatica, o meglio per la scienza in generale, durante il biennio, mi hanno formato e sicuramente aperto a davvero molte opportunità. Devo ammetterlo, quando al termine della scuola media ho dovuto scegliere il percorso che avrei affrontato per i successivi cinque anni, mi sono ritrovato di fronte ad un'entusiasmante passione per la matematica ed una forte indecisione tra il liceo scientifico e la mia attuale, o forse ora non più, scuola. Durante gli anni del biennio ero un po' titubante, non sicuro al 100% della scelta, ma con l'inizio del triennio d'informatica ho avuto la conferma di cui avevo bisogno. Questo percorso mi ha messo di fronte, oltre che ad un'ottima istruzione, ad una vasta serie di opportunità (un certo numero delle quali sono purtroppo andate perse a causa dell'emergenza sanitaria che stiamo vivendo), che sono sempre stato pronto ad accogliere. Raramente mi sono infatti tirato indietro di fronte a progetti interni o esterni o tirocini proposti dalla scuola, che mi hanno permesso di fare esperienza, dimostrare a me stesso cosa sono in grado di fare e di mettermi in contatto con realtà che potrebbero far parte del mio futuro. C'è in verità da dire che sono uno che punta in alto, addirittura sembra che stia maturando in me l'idea di un futuro tentativo per entrare nel progetto Neuralink di Elon Musk (lo so, mi sia concesso sognare), ma di certo tutte le realtà che ho conosciuto grazie al mio percorso potranno essere un ottimo punto di partenza.
Inoltre durante la durata di questo triennio, in quinta principalmente, mi sono trovato di fronte a svariate situazioni, scolastiche e non, che mi hanno fatto prendere sempre più coscienza di quella che è la vita. Ho sicuramente ancora molta strada da fare, ma sento di aver acquisito un certo livello di maturità, di consapevolezza, che mi mostrano la realtà in modo sempre più profondo. Penso che una della principali di-

mostrazioni di ciò, se non la più importante, sia il mio relativamente recente interesse per la letteratura e la storia, prima lasciate in secondo piano dalla mia mente maggiormente scientifica, ma di cui ora colgo l'importanza e il fascino. Mi ritengo molto fortunato dunque di aver trattato anche queste materie durante il mio percorso, che hanno contribuito in modo importante tanto quanto le altre a rendermi la persona che oggi sono e di cui vado fiero, non per vanità ma per una trovata felicità nella mia soddisfazione personale, di essere.

Questo progetto è stato sicuramente impegnativo per me, mi sono messo alla prova, e sono molto soddisfatto del risultato ottenuto. Ho utilizzato al meglio le mie competenze in ambito informatico, frutto di questi tre anni d'esplorazione di questo mondo e d'impegno personale. Sono molto soddisfatto di come sono riuscito a combinare tante tecnologie, che spaziano da moderne architetture a semplici librerie Javascript, in un unico ordinato ed equilibrato, almeno a mio avviso, progetto.
Lavorando a progetti di questo genere noto sempre di più quanto l'informatica sia vasta, e a quanto tutte le competenze ottenute grazie alla diverse materie inevitabilmente s'incontrino e necessitino le une delle altre. Mi sono imbattuto in diversi ostacoli durante lo svolgimento del progetto, mi sono ritrovato di fronte a molti problemi, ma che con forte motivazione e grazie all'approccio che la mia formazione mi garantisce sono sempre riuscito a risolvere, con più o meno difficoltà a seconda dei casi. Una cosa che di certo ho imparato in questi tre anni, e di cui sono definitivamente consapevole grazie a questo progetto, è che nel mondo moderno non si può mai smettere di imparare. Penso che l'informatico non sia quello che conosce tutte le importanti tecnologie a memoria, ma piuttosto una figura in grado, ogni giorno, di trovarsi di fronte ad un framework totalmente nuovo ed imparare ad utilizzarlo, senza aver bisogno di farselo spiegare da un professore. L'informatica non dev'essere un mondo composto da esperti di specifiche tecnologie, ma un mondo di persone con una mentalità volta ad affrontare ogni giorno nuove situazioni e strumenti, con un approccio portatore del lifelong learning.

Concludo parlando del mio futuro, per ora incline allo studio della fisica, che da anni mi appassiona. Penso che tutto ciò che ho imparato in questi anni mi sarà molto utile sia durante il mio percorso universitario che, e in questo caso ancor di più, nel mio futuro lavorativo. Non solo per le competenze tecniche acquisite, ma anche e soprattutto per la mentalità che ora guida sia il mio pensiero che il mio approccio tecnico.
Anche se solo una piccola parte di loro leggerà queste frasi, tengo a ringraziare tutte le persone che mi hanno seguito in questi anni, che hanno trasmesso in me quella,

anche se piccola, parte di loro che ha contribuito ad essere ciò che sono. Chiudo questo percorso a testa alta, felice e soddisfatto. Sono pronto per il mio futuro.

Lo studente Rocca Patrick

7

Bibliografia

Architetture

Microservices - Wikipedia: https://en.wikipedia.org/wiki/Microservices
Service-oriented architecture - Wikipedia: https://it.wikipedia.org/wiki/Service-oriented_architecture
Architetture SOA: https://www.html.it/pag/17183/architetture-soa/
What are Microservices? Code Examples, Best Practices, Tutorials and More: https://stackify.com/what-are-microservices/
Microservices Architecture: Il Pattern Architetturale Emergente Per Le Grandi Applicazioni Moderne: http://losviluppatore.it/microservices-architecture-il-pattern-architetturale-emergente-per-le-grandi-applicazioni-moderne/
Monolith, SOA, Microservices, or Serverless: https://rubygarage.org/blog/monolith-soa-microservices-serverless

AWS

AWS docs: https://docs.aws.amazon.com/
Getting started with AWS Lambda - Playlist YouTube: https://www.youtube.com/watch?v=fSUEk6iMW88
AWS Lambda Layers Tutorial - Managing Python libraries in a better way - YouTube: https://www.youtube.com/watch?v=3BH79Uciw5w

Sympy

Sympy docs: https://docs.sympy.org/latest/index.html

DMZ

Libro di testo di sistemi e reti - Internet Working (Elena Baldino, Renato Rondano, Antonio Spano, Cesare Iacobelli)

HTTP e HTTPS

HyperText Transfer Protocol - Wikipedia: https://it.wikipedia.org/wiki/Hypertext_Transfer_Protocol
HTTPS - Wikipedia: https://it.wikipedia.org/wiki/HTTPS

Spring Boot

Spring Boot docs: https://devdocs.io/spring_boot/
Spring Security docs: https://docs.spring.io/spring-security/
Spring Boot and Spring Security tutorials: https://www.baeldung.com
Tutorial vari su YouTube

Angular

Angular official site: https://angular.io/
Angular Material official site: https://material.angular.io/
Angular Data Binding: https://www.digitalocean.com/community/tutorials/angular-data-binding-angular
Angular Directives: https://www.sitepoint.com/practical-guide-angular-directives/
Enable CORS in Angular: https://www.positronx.io/handle-cors-in-angular-with-proxy-configuration/
Tutorial vari su YouTube

MathJax

MathJax official site: https://www.mathjax.org/
MathJax docs: https://docs.mathjax.org/en/latest/index.html

LaTeX.js

LaTeX.js official site: https://latex.js.org/